



**POLITECNICO
SETÚBAL**

Escola Superior de Tecnologia de Setúbal

PWDAM

EasyCharge

Projeto PCE

Matheus Santana - 2025130281

Rodrigo Fonseca - 2025168899

Rodrigo Belchior - 2025159599

Rogério Pereira - 2025138195

Setúbal, 13 de Abril de 2026

Escola Superior de Tecnologia de Setúbal

PWDAM

EasyCharge

Projeto PCE

Matheus Santana - 2025130281

Rodrigo Fonseca - 2025168899

Rodrigo Belchior - 2025159599

Rogério Pereira - 2025138195

Docentes orientadores:

José Canelas

Bernardo Milheiro

Manuel Ferreira

Manuel Ramos

Svetlana Chemetova

Relatório de Projeto Final

Setúbal, 13 de Abril de 2026

Resumo

O presente trabalho descreve o desenvolvimento do sistema EasyCharge, uma solução concebida para o armazenamento e carregamento de trotinetes elétricas, com capacidade de monitorização e controlo local e remoto. O sistema integra uma arquitetura composta por uma placa Basys2, responsável pela implementação da lógica digital e da máquina de estados, e por um microcontrolador ESP32, encarregado da comunicação entre o hardware, o computador e o smartphone.

A solução inclui ainda uma aplicação de controlo, mecanismos de comunicação entre módulos e armazenamento de dados recorrendo a CloudDB e TinyDB. Ao longo do projeto foram definidos os requisitos funcionais e não funcionais, bem como a arquitetura do sistema, permitindo estruturar de forma clara a solução proposta. Foram também abordadas as tecnologias utilizadas, a implementação do firmware e a comunicação entre os diferentes componentes.

Os resultados obtidos demonstram a viabilidade da solução proposta enquanto protótipo académico, evidenciando a integração entre hardware e software e a aplicação prática de conhecimentos adquiridos nas áreas de sistemas digitais, microcontroladores, comunicações e desenvolvimento de aplicações. Apesar das limitações identificadas, o sistema apresenta potencial de evolução futura, nomeadamente ao nível da escalabilidade, segurança e integração com sensores reais.

Palavras-chave: EasyCharge, trotinetes elétricas, cacifos inteligentes, ESP32, Basys2, IoT, monitorização.

Abstract

This work describes the development of the EasyCharge system, a solution designed for the storage and charging of electric scooters, with local and remote monitoring and control capabilities. The system integrates an architecture composed of a Basys2 board, responsible for implementing the digital logic and the state machine, and an ESP32 microcontroller, responsible for communication between the hardware, the computer, and the smartphone.

The solution also includes a control application, communication mechanisms between modules, and data storage using CloudDB and TinyDB. Throughout the project, the functional and non-functional requirements, as well as the system architecture, were defined, allowing the proposed solution to be clearly structured. The technologies used, the firmware implementation, and the communication between the different components were also addressed.

The results obtained demonstrate the feasibility of the proposed solution as an academic prototype, highlighting the integration between hardware and software and the practical application of knowledge acquired in the fields of digital systems, microcontrollers, communications, and application development. Despite the identified limitations, the system shows potential for future improvement, particularly in terms of scalability, security, and integration with real sensors.

Keywords: EasyCharge, electric scooters, smart lockers, ESP32, Basys2, IoT, monitoring.

Abreviaturas e Siglas

ADC	Conversor Analógico-Digital
App	Aplicação
Bluetooth	Tecnologia de comunicação sem fios de curto alcance
ESP32	Microcontrolador com conectividade Wi-Fi e Bluetooth integrada
FPGA	<i>Field Programmable Gate Array</i>
GPIO	<i>General Purpose Input/Output</i>
HTTP	<i>Hypertext Transfer Protocol</i>
IoT	<i>Internet of Things</i>
LCD	<i>Liquid Crystal Display</i>
LED	<i>Light Emitting Diode</i>
MQTT	<i>Message Queuing Telemetry Transport</i>
PWM	<i>Pulse Width Modulation</i>
SRAM	<i>Static Random Access Memory</i>
UART	<i>Universal Asynchronous Receiver-Transmitter</i>
USB	<i>Universal Serial Bus</i>
VHDL	<i>VHSIC Hardware Description Language</i>
Wi-Fi	<i>Wireless Fidelity</i>

Índice

1	INTRODUÇÃO	2
2	ANÁLISE E PLANEAMENTO	3
2.1	Definição do Problema	3
2.2	Objetivos do Projeto	3
2.2.1	Objetivos Específicos	4
2.3	Casos de Uso e Público Alvo	4
2.3.1	Casos de Uso	4
2.3.2	Público-Alvo	7
2.4	Requisitos Funcionais e Não Funcionais	7
2.4.1	Requisitos Funcionais	7
2.4.2	Requisitos Não Funcionais	8
3	ARQUITETURA E TECNOLOGIAS	10
3.1	Arquitetura IoT	10
3.1.1	Camada de Aplicação	10
3.1.2	Camada de Comunicação	10
3.1.3	Camada de Gateway / Middleware	11
3.1.4	Camada de Perceção / Dispositivo	11
3.2	Diagrama do Sistema	13
3.3	Protocolos de Comunicação	14
3.3.1	Sistema	14
3.3.2	Comunicação	15
3.3.2.1	Comunicação entre dispositivos	15
3.3.3	Ligações	16
3.3.4	Fiabilidade	16
3.4	Escolha da Aplicação e Justificação de Tecnologia	16
3.4.1	Utilização do ESP32	16
3.4.2	Utilização do Basys2	17
3.4.3	Não utilização do Arduino	17
3.4.4	Integração das Tecnologias	17
4	IMPLEMENTAÇÃO DO DISPOSITIVO (IOT)	18
4.1	Hardware (seleção)	18
4.1.1	Microcontrolador principal - ESP32	18
4.1.2	Tabela comparativa entre ESP32 e Arduino Uno	19

4.1.3	Placa Basys2	20
4.1.4	Arquitetura da Basys2	20
4.1.5	Ligações entre o ESP32 e a Basys2	21
4.1.6	Comunicação com o computador	22
4.1.7	Comunicação com o smartphone	22
4.1.8	Processo de comunicação de dados	22
4.1.9	Componentes auxiliares	22
4.2	Circuito	23
4.3	Firmware (leitura de sinais e envio de dados)	24
4.3.1	Funcionamento do Firmware	25
4.3.2	Possíveis extensões do sistema	25
4.3.3	Envio de Dados	25
4.3.3.1	Comunicação Basys2 → ESP32	25
4.3.3.2	Tipo de Dados	26
4.3.3.3	Envio para o sistema	26
4.3.4	Código do ESP32	26
4.3.5	Código da Basys2	29
4.3.5.1	Ficheiro de implementação .ucf	29
5	APLICAÇÃO E DEPLOYMENT	31
5.1	Base de Dados	31
5.1.1	CloudDB	31
5.1.2	TinyDB	32
5.1.3	Fluxo de Dados	32
5.1.4	Segurança	32
5.1.5	Limitações	33
5.2	Dashboard ou App	33
5.3	Visualização e Conteúdo	34
5.3.1	Página de Carregamento	35
5.3.2	Página de Histórico	36
5.3.3	Página do Mapa	37
5.3.4	Página de Perfil	38
5.3.5	Funcionalidades Complementares	39
6	GESTÃO DE CÓDIGO NO BITBUCKET	41
6.1	Comparação entre Bitbucket, GitHub e GitLab	41
7	ANÁLISE DOS SPRINTS	43

8	TESTES, RESULTADOS E CONCLUSÃO	45
8.1	Testes e Resultados Obtidos	45
8.1.1	Comunicação	45
8.1.2	Abertura e Fecho dos Cacifos	45
8.1.3	Ocupação e disponibilidade	46
8.1.4	Interface com o utilizador	46
8.2	Propostas de manutenção	46
8.2.1	Manutenção Preventiva	46
8.2.2	Manutenção Corretiva	46
8.2.3	Manutenção de Software	46
8.2.4	Monitorização do Sistema	47
8.3	Limitações	47
8.4	Melhorias Futuras	47
9	CONCLUSÃO	49
10	BIBLIOGRAFIA	50

Índice de Figuras

Figura 1 – Arquitetura IoT do sistema organizada por camadas.	12
Figura 2 – Diagrama de funcionamento do sistema EasyCharge.	14
Figura 3 – Arquitetura interna do sistema implementado na Basys2	21
Figura 4 – Montagem prática do sistema EasyCharge.	24
Figura 5 – Página Principal	34
Figura 6 – Página de Carregamentos	36
Figura 7 – Histórico de cargas	37
Figura 8 – Página de mapa de carregadores	38
Figura 9 – Página de perfil	39
Figura 10 – Sprint Testes, Resultados e Conclusões	43
Figura 11 – Sprint Análise e Planeamento	44

Índice de Tabelas

Tabela 2 – Comparação entre o ESP32 e o Arduino Uno	19
Tabela 3 – Exemplo de interpretação dos sinais enviados da Basys2 para o ESP32 . . .	26

1 Introdução

O projeto EasyCharge consiste no desenvolvimento de um sistema de cacifos inteligentes para armazenamento e carregamento de trotinetes elétricas. O objetivo é criar uma solução prática que permita aos utilizadores guardar a sua trotinete num local seguro enquanto esta carrega a bateria.

Com o aumento da utilização de meios de mobilidade elétrica nas cidades, torna-se importante disponibilizar infraestruturas que facilitem o carregamento destes veículos. O sistema EasyCharge pretende responder a essa necessidade através de um sistema composto por hardware, software e comunicação entre dispositivos.

Ao longo deste relatório, será apresentado de forma detalhada todo o processo de desenvolvimento do projeto. Inicialmente, é abordado o enquadramento do problema e os objetivos do sistema. De seguida, é apresentado o desenvolvimento técnico, onde são descritos os principais componentes do sistema, incluindo a arquitetura, o circuito, o firmware, a base de dados e o protocolo de comunicações.

O documento inclui ainda um capítulo dedicado às ferramentas de apoio ao desenvolvimento, como o Jira, utilizado para a gestão de tarefas, e o Bitbucket, utilizado para o controlo de versões do projeto.

Por fim, o relatório termina com uma conclusão, onde são analisados os resultados obtidos, identificadas as limitações do projeto e apresentadas possíveis melhorias e evoluções futuras do sistema.

2 Análise e Planeamento

Neste capítulo é apresentada a fase de análise e planeamento do projeto, fundamental para a definição estruturada da solução a desenvolver. Inicialmente, procede-se à identificação e caracterização do problema, de forma a enquadrar a necessidade da implementação do sistema. De seguida, são estabelecidos os principais objetivos do projeto, clarificando os resultados que se pretendem alcançar. Posteriormente, são analisados os casos de uso e o público-alvo, permitindo compreender o contexto de utilização da solução proposta e as necessidades dos seus utilizadores. Por fim, são definidos os requisitos funcionais e não funcionais, essenciais para orientar o desenvolvimento das várias componentes do sistema.

2.1 Definição do Problema

O aumento na utilização das trotinetes elétricas como meio de mobilidade urbana sustentável trouxe à tona novas necessidades em termos de infraestruturas de apoio. Mais concretamente, isso engloba o estacionamento seguro e o carregamento de energia dos referidos veículos. Locais onde costumamos frequentar ainda não oferecem uma solução que possa dar-nos, na mesma medida, segurança física e carregamento de energia.

As pessoas que utilizam trotinetes frequentemente colocam-nas em locais improvisados que não dão proteção contra o furto ou vandalismo e não fornecem fácil acesso à energia. Isto não só coloca em risco a segurança do equipamento, como também reduz a eficiência do seu uso diário. Diante disso, podemos listar alguns dos principais problemas :

- Não existem locais apropriados e seguros para guardar as trotinetes elétricas;
- Não há pontos de carregamento devidamente organizados e acessíveis;
- O furto e os atos de vandalismo são riscos muito elevados;
- Não é possível monitorizar o nível de carga das baterias.

Há ausência de mecanismos que permitam o registo e controlo do uso das trotinetes elétricas.

2.2 Objetivos do Projeto

Com este projeto pretende-se desenvolver um sistema integrado de controlo de um equipamento, concretamente um sistema com quatro compartimentos destinados ao armazenamento e carregamento de trotinetas elétricas. O objetivo consiste na implementação de uma solução completa que permita o controlo local e remoto do equipamento, integrando hardware programável,

microcontrolador, aplicação móvel e mecanismos de comunicação entre todos os módulos do sistema.

2.2.1 Objetivos Específicos

- Desenvolver um sistema de controlo baseado numa máquina de estados microprogramada, implementada numa placa de lógica programável, com o objetivo de simular o funcionamento do equipamento;
- Implementar, no ESP32, a interface entre o sistema lógico e os restantes dispositivos, assegurando a leitura e envio de sinais digitais, bem como as comunicações com o computador e com o smartphone;
- Desenvolver uma aplicação móvel para Android, recorrendo ao App Inventor, que permita o controlo remoto do equipamento, a visualização do seu estado e a interação com os diferentes cacifos;
- Implementar mecanismos de armazenamento de dados, recorrendo a CloudDB e TinyDB, para suportar funcionalidades como autenticação, histórico de carregamentos e registo de informação associada ao utilizador;
- Integrar todos os componentes do sistema, garantindo a comunicação entre hardware e software e validando o funcionamento do equipamento simulado;
- Aplicar, de forma prática, os conhecimentos adquiridos nas diferentes unidades curriculares do curso.

2.3 Casos de Uso e Público Alvo

Nesta secção são apresentados os principais utilizadores do sistema EasyCharge, bem como as funcionalidades fundamentais associadas à sua utilização. Numa primeira fase, são descritos os principais casos de uso do sistema, com o objetivo de evidenciar as interações possíveis entre os utilizadores, o sistema e a entidade gestora. Esta análise permite compreender de forma estruturada os requisitos funcionais da solução proposta e o modo como o sistema responde às necessidades identificadas. Em seguida, identifica-se o público-alvo a que a solução se destina, tendo em conta o contexto de utilização e as necessidades dos potenciais utilizadores.

2.3.1 Casos de Uso

- **Caso 1 – Autenticar Utilizador**

Ator: Utilizador

Descrição: O utilizador autentica-se no sistema para poder utilizar um cacifo.

Pré-condição: Sistema ligado e operacional.

Fluxo principal:

- Utilizador introduz as suas credenciais na aplicação.
- Sistema valida os dados.
- Acesso autorizado.

• **Caso 2 – Reservar Cacifo**

Ator: Utilizador

Descrição: Permite reservar um dos 4 lugares disponíveis.

Fluxo principal:

- Utilizador consulta disponibilidade.
- Seleciona um lugar livre.
- Sistema confirma reserva.
- Estado do lugar muda para “Reservado”.

• **Caso 3 – Abrir Cacifo**

Ator: Utilizador

Descrição: Abre o compartimento para guardar ou levantar a trotinete.

Fluxo principal:

- Utilizador seleciona o cacifo.
- Sistema envia comando ao ESP32.
- ESP32 ativa sinal.
- Cacifo abre.
- Evento é registado no sistema.

• **Caso 4 – Iniciar Carregamento**

Ator: Utilizador

Descrição: Inicia o carregamento da trotinete após colocação no cacifo.

Fluxo principal:

- Sistema deteta trotinete no interior.
- Utilizador seleciona “Iniciar carregamento”.
- Sistema ativa circuito de carregamento.
- Estado passa para “Em carregamento”.

- **Caso 5 – Terminar Carregamento**

Ator: Utilizador

Descrição: Finaliza o carregamento antes ou após carga completa.

Fluxo principal:

- Utilizador seleciona “Terminar carregamento”.
- Sistema desativa carregamento.
- Evento é registado.

- **Caso 6 – Consultar Estado do Cacifo**

Ator: Utilizador

Descrição: Permite visualizar:

- Disponível
- Reservado
- Ocupado
- Em carregamento
- Com erro

Interface: Aplicação para telemóvel.

- **Caso 7 – Registo de Eventos**

Ator: Sistema

Descrição: Regista automaticamente:

- Abertura/fecho
- Início/fim de carregamento
- Erros
- Tentativas inválidas
- Falhas de comunicação

- **Caso 8 – Monitorização Remota**

Ator: Entidade gestora

Descrição: Permite acompanhar:

- Número de utilizações
- Tempo médio de carregamento
- Histórico de eventos
- Estado dos 4 lugares

2.3.2 Público-Alvo

O sistema EasyCharge destina-se a utilizadores de trotinetes elétricas como meio de transporte no seu dia a dia, nomeadamente estudantes universitários, funcionários e professores, bem como utilizadores urbanos em geral. Estes utilizadores necessitam de um local seguro onde possam carregar a sua trotinete enquanto permanecem nas suas atividades académicas ou profissionais, garantindo assim maior autonomia no regresso. Procuram ainda uma solução prática, simples e rápida de utilizar, preferencialmente com possibilidade de controlo e monitorização remota através de smartphone, que lhes permita acompanhar o estado do cacifo e do carregamento de forma cómoda e eficiente.

2.4 Requisitos Funcionais e Não Funcionais

A implementação do sistema de controlo de cacifos para guardar e carregar trotinetas elétricas envolve várias componentes. Para garantir que todas estas partes funcionam de forma integrada e que o resultado final cumpre o objetivo do projeto, é essencial definir os requisitos do sistema. Esses requisitos servem como base para a especificação, desenvolvimento e testes, permitindo evitar ambiguidades e assegurar que o equipamento pode ser comandado, monitorizado e registado/armazenado de forma correta, tanto localmente (PC), como remotamente (smartphone).

2.4.1 Requisitos Funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer, ou seja, as funcionalidades e serviços que tem de disponibilizar ao utilizador. De seguida, apresentam-se os principais requisitos funcionais do sistema.

Controlo do equipamento:

- O sistema deve permitir controlar um cacifo com 4 compartimentos para guardar e carregar trotinetas elétricas.
- O sistema deve permitir a abertura e o fecho individual de cada compartimento.
- O sistema deve indicar o estado de cada compartimento.

Comunicação entre componentes:

- O sistema deve assegurar a comunicação entre o computador e o ESP32 através de porta série virtual USB.
- O sistema deve assegurar a comunicação entre o ESP32 e a Basys2 através de sinais digitais.
- O sistema deve permitir a comunicação entre o smartphone e o sistema por Bluetooth.

- O sistema deve usar mensagens estruturadas para a troca de dados entre módulos.

Software do computador:

- O software do computador deve permitir enviar comandos para o equipamento.
- O software deve apresentar o estado atual do sistema.
- O software deve manter um registo temporal dos eventos.

Software do ESP32:

- O ESP32 deve receber comandos enviados pelas aplicações.
- O ESP32 deve gerar sinais digitais de controlo para a Basys2.
- O ESP32 deve receber sinais de estado da Basys2 e convertê-los em mensagens para o computador e/ou smartphone.

Software do smartphone:

- A aplicação Android deve permitir enviar comandos ao equipamento.
- A aplicação deve permitir visualizar informação de estado do sistema.

Máquina de estados microprogramada:

- A máquina de estados implementada na Basys2 deve simular o funcionamento do cacifo.
- A Basys2 deve receber comandos através de entradas digitais.
- A Basys2 deve enviar sinais de estado e eventos ao ESP32.
- O estado do sistema deve poder ser visualizado localmente com LEDs e displays.

2.4.2 Requisitos Não Funcionais

Os requisitos não funcionais descrevem como o sistema deve ser implementado e que condições/qualidades deve cumprir. De seguida, apresentam-se os principais requisitos funcionais do sistema.

Desempenho:

- O sistema deve responder aos comandos em tempo quase real.
- A comunicação entre módulos deve apresentar baixa latência.

Fiabilidade:

- O sistema deve garantir consistência entre comandos enviados e estados apresentados.
- Em caso de falha de comunicação, o sistema deve manter um estado válido.

Usabilidade:

- As interfaces do computador e do smartphone devem ser simples e intuitivas.
- O utilizador deve conseguir identificar facilmente o estado dos compartimentos.

Compatibilidade:

- O software do computador deve ser compatível com Windows ou Linux.
- A aplicação móvel deve ser compatível com Android.
- O sistema deve suportar comunicação por USB e Bluetooth, conforme a arquitetura adotada.

Manutenibilidade:

- A solução deve ser modular, separando computador, ESP32, smartphone e Basys2.
- O protocolo de comunicação deve estar devidamente documentado.

Registo e monitorização:

- O sistema deve armazenar um histórico temporal de eventos no computador.

3 Arquitetura e Tecnologias

Neste capítulo apresenta-se a arquitetura do sistema e as principais tecnologias adotadas para o desenvolvimento da solução proposta. Numa primeira fase, é descrita a arquitetura IoT do sistema, organizada por camadas, de forma a evidenciar a estrutura e a interação entre os diferentes níveis funcionais. Em seguida, é apresentado o diagrama do sistema, permitindo uma visão global dos seus componentes e das ligações existentes entre hardware e software. Posteriormente, são abordados os protocolos de comunicação utilizados, fundamentais para assegurar a troca de informação entre os vários módulos do sistema. Por fim, é justificada a escolha das aplicações e das tecnologias adotadas, tendo em conta os objetivos do projeto, os requisitos definidos e a adequação de cada solução ao contexto de implementação.

3.1 Arquitetura IoT

A solução desenvolvida pode ser organizada segundo uma arquitetura por camadas, o que permite representar de forma clara as funções de cada componente do sistema. Esta abordagem facilita a compreensão da interação entre a BASYS2, o ESP32, os meios de comunicação e as aplicações de utilizador. Assim, o projeto pode ser dividido em quatro camadas principais: Aplicação, Comunicação, Gateway/Middleware e Perceção/Dispositivo, como se observa na Figura 1.

3.1.1 Camada de Aplicação

A camada de aplicação integra as interfaces com o utilizador. Esta camada inclui o software desenvolvido em C para computador, responsável por gerir as comunicações, permitir a execução de comandos, visualizar o estado do equipamento e manter um registo temporal de eventos, bem como a aplicação Android desenvolvida em AppInventor para controlo e monitorização remota. Deste modo, esta camada representa o ponto de contacto entre o utilizador e o sistema.

3.1.2 Camada de Comunicação

Esta camada corresponde aos mecanismos de transporte de informação entre as aplicações de utilizador e o sistema físico. A comunicação entre o computador e o gateway é realizada através de uma porta série virtual via USB, enquanto a comunicação com o smartphone pode ser feita por Bluetooth, também recorrendo a uma porta série virtual. Assim, esta camada é responsável apenas pelo encaminhamento das mensagens, comandos e estados entre os diferentes elementos da arquitetura.

3.1.3 Camada de Gateway / Middleware

Esta camada é implementada pelo ESP32, que funciona como elemento intermédio entre a Basys2 e as aplicações de utilizador. Cabe-lhe receber os comandos enviados a partir do computador ou do smartphone, convertê-los nos sinais de controlo necessários ao funcionamento do equipamento simulado e assegurar a comunicação de dados com essas plataformas. Quando necessário, o ESP32 recebe também os sinais vindos da BASYS2 e gera as mensagens adequadas para que o computador registre eventos e informe o utilizador. Deste modo, esta camada desempenha a função de gateway local, realizando a adaptação entre a interface digital com o equipamento e as comunicações série usadas pelas aplicações.

3.1.4 Camada de Percepção / Dispositivo

Esta camada corresponde ao equipamento simulado e à sua interface física de entradas e saídas digitais. É implementada na placa BASYS2, onde é executada a máquina de estados microprogramada responsável por simular o comportamento do cacifo com 4 lugares para guardar e carregar trotinetas elétricas. A comunicação com o ESP32 é feita através de sinais digitais, pelos quais a BASYS2 recebe comandos e envia informação de estado e eventos. Integram-se ainda nesta camada os LEDs, os displays de 7 segmentos e um LCD, utilizados para visualizar localmente o funcionamento do equipamento simulado, podendo também ser usados botões e interruptores para inicialização ou interação com a máquina de estados.

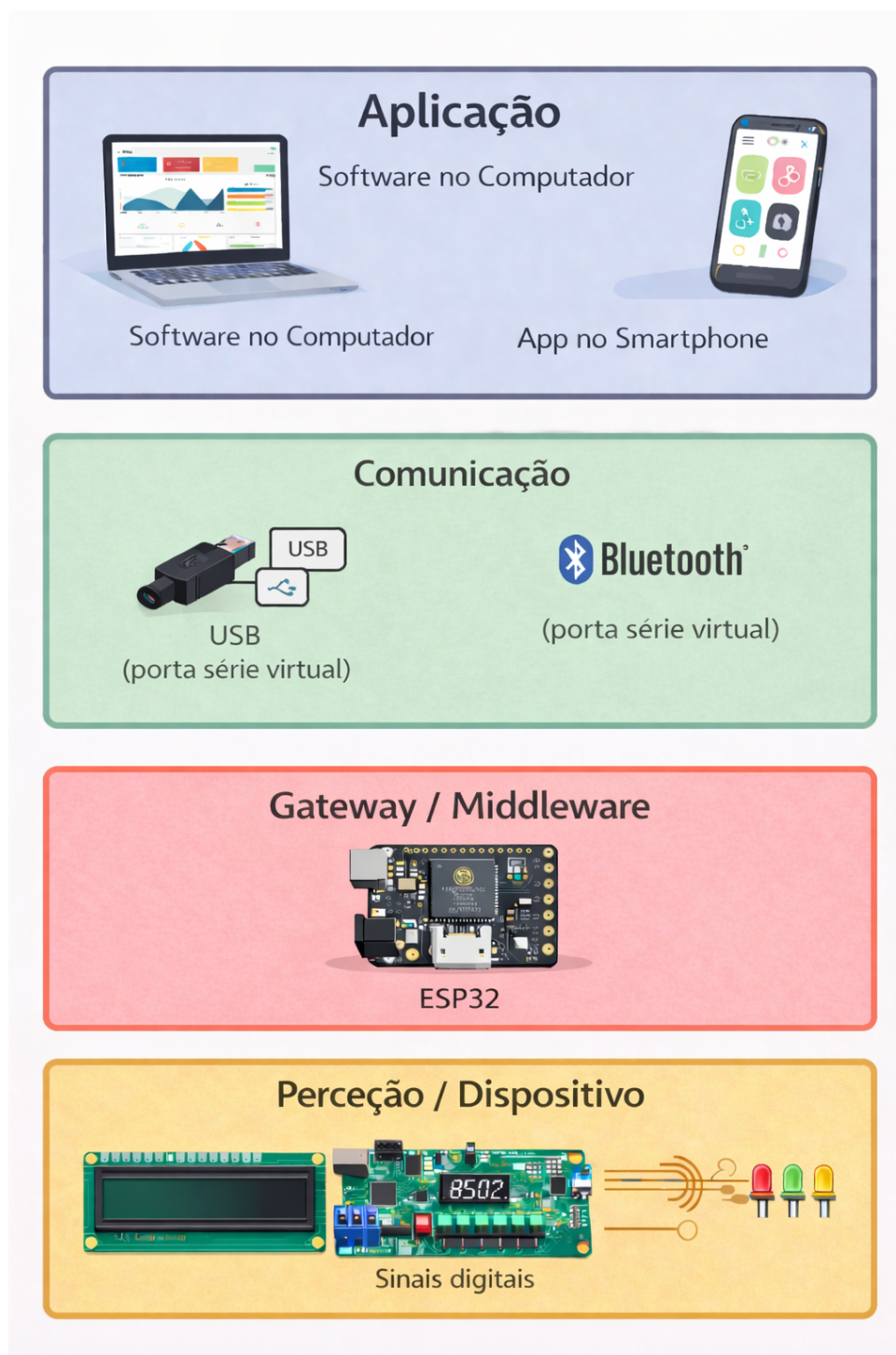


Figura 1 – Arquitetura IoT do sistema organizada por camadas.

A organização por camadas permite compreender de forma estruturada o papel de cada componente do sistema e a forma como se processa a interação entre hardware, software e utilizador.

3.2 Diagrama do Sistema

O diagrama apresentado na Figura 2, representa o fluxo de funcionamento do sistema EasyCharge durante a utilização dos cacifos para armazenamento e carregamento de trotinetes elétricas.

Numa fase inicial, o sistema permanece em modo de espera até que um utilizador efetue o processo de autenticação. Após a introdução das credenciais, o sistema procede à respetiva validação. Caso o utilizador não esteja autenticado, é apresentada a mensagem de “Acesso negado” e o processo é interrompido. Se a autenticação for válida, o sistema verifica a disponibilidade de cacifos.

Se não existirem cacifos disponíveis, é apresentada a mensagem “Cacifos indisponíveis” e o fluxo termina. Caso exista disponibilidade, o sistema ativa o sinal digital, através do microcontrolador, para permitir a abertura do compartimento e aguarda a colocação da trotinete.

De seguida, o sistema verifica o estado associado à presença da trotinete. Caso não seja detetada qualquer trotinete, o estado do cacifo regressa a “Livre”. Se a presença for confirmada, o estado do cacifo é alterado para “Ocupado” e o evento é registado.

Após a deteção da trotinete no interior do compartimento, o sistema entra na fase de carregamento, permanecendo nesse estado até que o carregamento seja concluído ou terminado manualmente pelo utilizador. Quando o processo termina, o estado do cacifo é novamente atualizado para “Livre” e o evento de saída é registado, concluindo-se assim o ciclo de funcionamento.

Desta forma, o diagrama demonstra a sequência lógica de decisões, ações e verificações que garantem o correto funcionamento do sistema, incluindo o tratamento de erros e a atualização consistente dos estados dos cacifos.

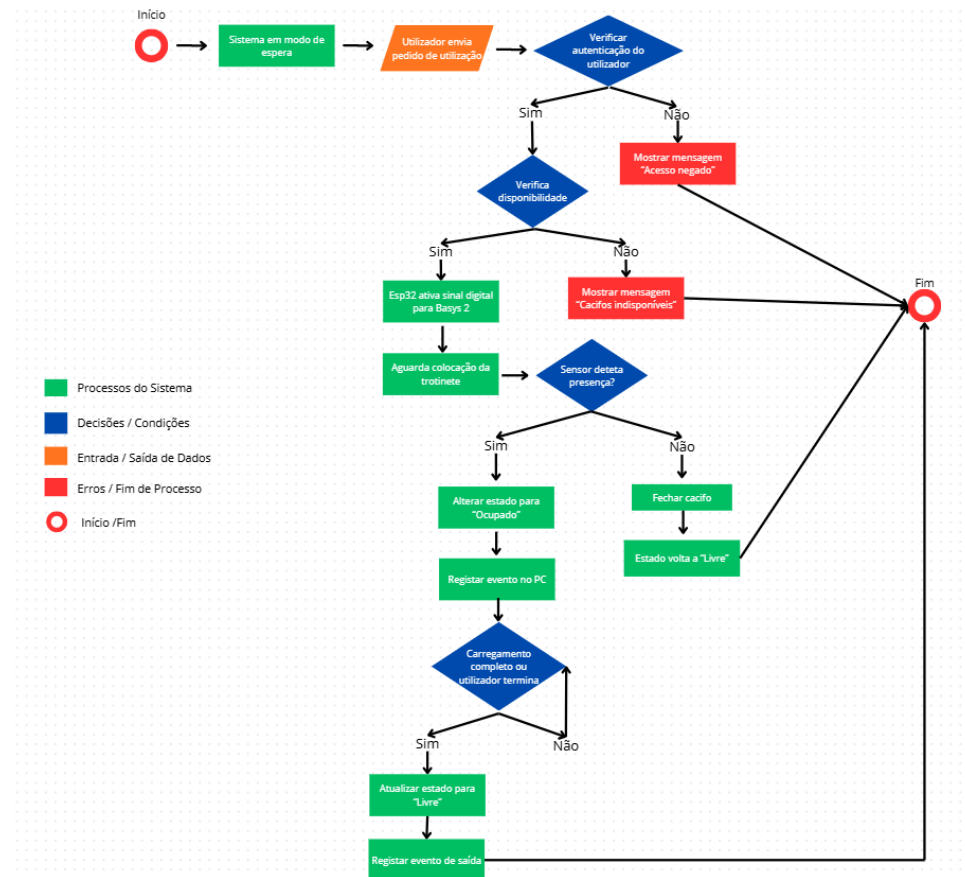


Figura 2 – Diagrama de funcionamento do sistema EasyCharge.

3.3 Protocolos de Comunicação

Nesta secção são apresentados os protocolos de comunicação adotados no sistema EasyCharge, fundamentais para assegurar a troca de informação entre os diferentes componentes da arquitetura. Uma vez que o sistema integra múltiplos dispositivos, nomeadamente smartphone, computador, ESP32 e placa Basys2, torna-se necessário definir de forma clara como são transmitidos os comandos, os estados e os eventos ao longo do seu funcionamento. A correta definição destes mecanismos de comunicação é essencial para garantir a coordenação entre hardware e software, bem como a fiabilidade e consistência do sistema.

3.3.1 Sistema

O EasyCharge constitui-se por uma arquitetura distribuída por smartphone, computador, ESP32 e placa Basys2, sendo necessário definir um protocolo de comunicações que assegure a troca consistente de comandos, estados e eventos entre todos os componentes. O computador funciona como o centro do sistema, recebendo pedidos do smartphone, enviando os comandos para o ESP32 e registando todos os eventos.

3.3.2 Comunicação

O smartphone e o sistema comunicam por *Bluetooth*, permitindo que o utilizador envie pedidos de autenticação, consulta de estado (ocupado ou livre), reserva de cacifo, abertura e controlo do carregamento. Cada comando recebido é validado pelo sistema, que responde com confirmação, erro ou envio do estado atual dos cacifos, garantindo controlo remoto e feedback imediato ao utilizador.

Para além da descrição geral da comunicação remota, importa referir que a ligação entre a aplicação móvel e o ESP32 é realizada por Bluetooth Classic, recorrendo ao perfil SPP (*Serial Port Profile*), que permite simular uma porta série sem fios. Quando o microcontrolador é iniciado, fica disponível para emparelhamento com o nome EasyCharge, sendo a ligação estabelecida a partir da aplicação móvel. A troca de informação é feita através de mensagens de texto simples, permitindo à aplicação enviar comandos e receber atualizações de estado em tempo real.

3.3.2.1 Comunicação entre dispositivos

No sistema EasyCharge, o processo de comunicação de dados inicia-se na placa Basys2, onde é implementada a lógica de funcionamento dos quatro lugares de armazenamento e carregamento. A Basys2 gera sinais digitais que representam o estado de cada lugar, nomeadamente a ocupação (através dos switches), o tempo de carregamento (contador) e o sinal de reset do sistema.

Estes sinais são lidos pelo ESP32, que atua como elemento intermédio entre a lógica digital implementada na placa e as aplicações de utilizador. O ESP32 interpreta os sinais recebidos e converte-os em mensagens compreensíveis, como por exemplo S3:OCUPADO ou S2:LIVRE, bem como mensagens completas de estado como S3:1:5 S2:0:0, onde cada valor representa a ocupação e o tempo de utilização de cada espaço.

A comunicação com o computador é realizada através de ligação USB e porta série virtual, permitindo apresentar o estado dos lugares, registar eventos e enviar comandos de controlo. Em paralelo, a comunicação com o smartphone é realizada por Bluetooth, possibilitando a monitorização em tempo real e o controlo remoto do sistema.

Desta forma, o fluxo de dados segue o percurso Basys2 → ESP32 → aplicações, sendo o ESP32 responsável pela adaptação entre sinais digitais de hardware e mensagens utilizadas pelas interfaces de utilizador.

Para além da receção de estados, o ESP32 também envia para a Basys2 sinais de controlo, como o reset do sistema, ativado a partir das aplicações. Esta comunicação bidirecional permite que os utilizadores interfiram diretamente no comportamento do sistema, garantindo a integração entre a componente lógica implementada em hardware e as interfaces de supervisão e controlo desenvolvidas.

3.3.3 Ligações

A ligação entre o ESP32 e a Basys2 FPGA Board é implementada através de sinais digitais diretos. A Basys2 executa a lógica principal do sistema, incluindo a máquina de estados dos cacifos, sendo responsável por determinar a ocupação de cada espaço com base nos switches e pela contagem do tempo de utilização.

O ESP32 atua como interface de comunicação, lendo os sinais digitais provenientes da Basys2 (ocupação e contadores) e convertendo-os em mensagens compreensíveis para as aplicações externas. Para além disso, o ESP32 pode enviar sinais de controlo simples, como o reset do sistema, permitindo a interação a partir do computador ou da aplicação móvel.

3.3.4 Fiabilidade

A fiabilidade do sistema baseia-se na simplicidade da comunicação digital entre os dispositivos e na atualização periódica dos estados. Como os dados são enviados continuamente pelo ESP32 (via USB e Bluetooth), qualquer alteração no estado dos cacifos é rapidamente refletida nas aplicações de utilizador.

Embora não tenha sido implementado um protocolo completo com confirmação de mensagens, a frequência de atualização garante consistência prática dos dados e funcionamento estável durante a utilização do sistema.

3.4 Escolha da Aplicação e Justificação de Tecnologia

Para a implementação do sistema foram utilizados dois dispositivos principais: o ESP32 e o Basys2, sendo cada um responsável por diferentes funções dentro do projeto.

A utilização conjunta destas tecnologias permite dividir o funcionamento do sistema entre controlo lógico digital e controlo inteligente com comunicação, tornando o projeto mais organizado e eficiente.

3.4.1 Utilização do ESP32

O ESP32 foi escolhido para realizar as funções de controlo geral e comunicação do sistema. Este microcontrolador permite:

- Comunicação por Bluetooth;
- Comunicação série com o computador por USB;
- Leitura dos sinais digitais recebidos da Basys2;
- Envio de informação para a aplicação móvel e para o software em C;
- Execução de comandos de controlo, como o reset do sistema.

O ESP32 foi selecionado por possuir conectividade sem fios integrada, boa capacidade de processamento e baixo custo, características que o tornam adequado para sistemas inteligentes com necessidade de comunicação entre hardware e aplicações de utilizador.

3.4.2 Utilização do Basys2

O Basys2 foi escolhido para implementar a lógica digital do sistema, através da programação da FPGA utilizando a linguagem VHDL.

Esta placa permite:

- Implementar circuitos digitais;
- Trabalhar com lógica combinatória e sequencial;
- Controlar sinais digitais de forma precisa;
- Aplicar conhecimentos de sistemas digitais.

A utilização do Basys2 justifica-se pela necessidade de implementar a lógica dos cacifos ao nível do hardware, incluindo contadores, estados e sinais digitais de controlo, permitindo também aplicar os conhecimentos adquiridos na unidade curricular.

3.4.3 Não utilização do Arduino

O Arduino não foi utilizado como plataforma principal neste projeto devido às suas limitações em comparação com o ESP32 e o Basys2. Apesar de ser uma solução simples e adequada para projetos básicos, o Arduino não possui conectividade Wi-Fi ou Bluetooth integrada, sendo necessário utilizar módulos adicionais para permitir a comunicação remota, o que aumentaria a complexidade do sistema. Além disso, apresenta menor capacidade de processamento e memória, o que poderia limitar o desempenho em tarefas que envolvem controlo simultâneo de vários componentes. Por outro lado, a utilização do Basys2 permitiu implementar a lógica digital do sistema ao nível do hardware, algo que não é possível fazer diretamente com o Arduino. Assim, a combinação entre ESP32 e Basys2 revelou-se mais adequada aos objetivos técnicos e funcionais do projeto.

3.4.4 Integração das Tecnologias

O funcionamento do sistema baseia-se na integração entre o ESP32 e o Basys2.

O ESP32 é responsável pela comunicação e controlo geral do sistema, enquanto o Basys2 é responsável pelo processamento lógico dos sinais digitais.

Esta divisão permite uma estrutura modular, onde cada dispositivo desempenha a função para a qual é mais adequado.

4 Implementação do Dispositivo (IoT)

Neste capítulo é apresentada a implementação do dispositivo IoT desenvolvido para o sistema EasyCharge, abordando as principais componentes de hardware e software responsáveis pelo seu funcionamento. Inicialmente, é descrita a seleção do hardware utilizado, tendo em conta as necessidades do projeto e as funções a desempenhar por cada elemento. De seguida, é apresentado o circuito implementado, evidenciando as ligações entre os diferentes componentes do sistema. Por fim, é abordado o firmware desenvolvido, responsável pela leitura dos sinais digitais do sistema, processamento da informação e envio de dados.

4.1 Hardware (seleção)

Nesta secção apresenta-se a seleção do hardware adotado para a implementação do sistema de controlo do cacifo automático com 4 lugares para armazenamento e carregamento de trotinetas elétricas. A escolha dos componentes foi feita tendo em conta os requisitos funcionais do projeto, nomeadamente a necessidade de comunicação com o computador e com o smartphone, bem como a implementação da lógica de controlo e da simulação do equipamento. Embora o enunciado proponha originalmente uma solução baseada em Arduino e Nexys2, optou-se pela utilização de ESP32 e Basys2, por constituírem uma alternativa funcionalmente equivalente e adequada aos objetivos pretendidos.

4.1.1 Microcontrolador principal - ESP32

Foi escolhido o ESP32 como unidade responsável pela interface entre o computador, o smartphone e a placa FPGA. As suas principais funções são:

- Receber comandos enviados pelo computador;
- Comunicar com a aplicação móvel através de Bluetooth;
- Gerar sinais simples de controlo para a FPGA, como o reset do sistema;
- Ler sinais de estado e eventos enviados pela FPGA;
- Reencaminhar informação para o computador ou para a aplicação móvel.

O ESP32 foi selecionado por apresentar várias vantagens relativamente a uma solução baseada num Arduino tradicional, tais como:

- Comunicação série USB para ligação ao computador;
- Bluetooth integrado, evitando a utilização de módulos externos;

- Número suficiente de GPIOs digitais para interface com a Basys2;
- Maior capacidade de processamento e flexibilidade.

Assim, o ESP32 desempenha o papel de interface de comunicação e controlo do sistema, assegurando a ligação entre a lógica digital implementada na FPGA e as aplicações de utilizador.

4.1.2 Tabela comparativa entre ESP32 e Arduino Uno

Característica	ESP32	Arduino Uno
Arquitetura	32-bit	8-bit
Processador	Xtensa LX6, 1 ou 2 núcleos, até 240 MHz	ATmega328P, 16 MHz
Memória SRAM	520 KB	2 KB
Memória Flash	Depende do módulo/placa; tipicamente superior à do Uno	32 KB
Wi-Fi incorporado	Sim	Não
Bluetooth incorporado	Sim	Não
GPIOs	Até 34 GPIOs programáveis (dependendo da variante)	14 pinos digitais
Entradas analógicas	Até 18 canais ADC (dependendo da variante)	6 entradas analógicas
PWM	Sim	Sim
Tensão lógica de funcionamento	3,3 V	5 V
Comunicação USB com computador	Sim, nas placas de desenvolvimento ESP32	Sim
Modos de baixo consumo	Sim: modem-sleep, light-sleep, deep-sleep e hibernação	Muito mais limitados
Deep sleep	Sim	Não como funcionalidade nativa comparável
Comunicação sem fios sem módulos externos	Sim	Não
Adequação para aplicações IoT	Elevada	Limitada sem hardware adicional

Tabela 2 – Comparação entre o ESP32 e o Arduino Uno

A Tabela 2 mostra que o ESP32 apresenta vantagens significativas face ao Arduino Uno, sobretudo ao nível da capacidade de processamento, memória disponível e integração de interfaces

de comunicação sem fios. O facto de incluir Wi-Fi e Bluetooth permite eliminar a necessidade de módulos externos, simplificando a arquitetura do sistema. Além disso, o suporte para modos de baixo consumo, incluindo deep sleep, constitui uma vantagem adicional em aplicações. Assim, a escolha do ESP32 revela-se mais adequada aos requisitos do projeto, mantendo a função de interface entre o computador, o smartphone e a FPGA.

4.1.3 Placa Basys2

A placa Basys2 foi utilizada como componente responsável pela simulação do comportamento do equipamento controlado. Nesta placa foi implementada a lógica digital que representa o funcionamento dos quatro lugares do sistema EasyCharge, recorrendo a uma máquina de estados e a módulos auxiliares de contagem e visualização. A Basys2 permite, assim, simular localmente o comportamento dos cacifos e observar o seu estado através dos elementos físicos disponíveis na própria placa.

No contexto desta implementação, os switches da Basys2 foram utilizados para simular a ocupação dos lugares, representando a entrada ou saída de uma trotinete em cada compartimento. Sempre que um switch correspondente a um lugar é ativado, o sistema considera esse lugar ocupado e inicia o processo de carregamento simulado através de um contador. Os LEDs associados funcionam como indicador visual imediato do estado de cada lugar, permitindo verificar localmente se o compartimento se encontra ativo ou inativo. Os displays de 7 segmentos foram utilizados para apresentar a evolução do carregamento, através de contadores que incrementam progressivamente até ao valor máximo definido. O botão de reset global permite reiniciar todos os contadores e repor o sistema ao estado inicial.

O funcionamento do sistema na Basys2 ocorre da seguinte forma: quando um lugar é ativado por um switch, o LED correspondente acende e o contador associado começa a evoluir no display. Quando esse lugar é desativado, o LED apaga, o display deixa de apresentar contagem e o respetivo contador é reiniciado. Em caso de reset global, todos os contadores regressam ao valor inicial, independentemente do estado anterior de cada lugar. Desta forma, a Basys2 permite visualizar de forma simples e local a ocupação, o processo de carregamento e o reinício do sistema, constituindo a base física de simulação do protótipo desenvolvido.

4.1.4 Arquitetura da Basys2

A lógica implementada na Basys2 é composta por vários módulos, incluindo contadores, divisores de clock e módulos de visualização, responsáveis por simular o comportamento dos cacifos.

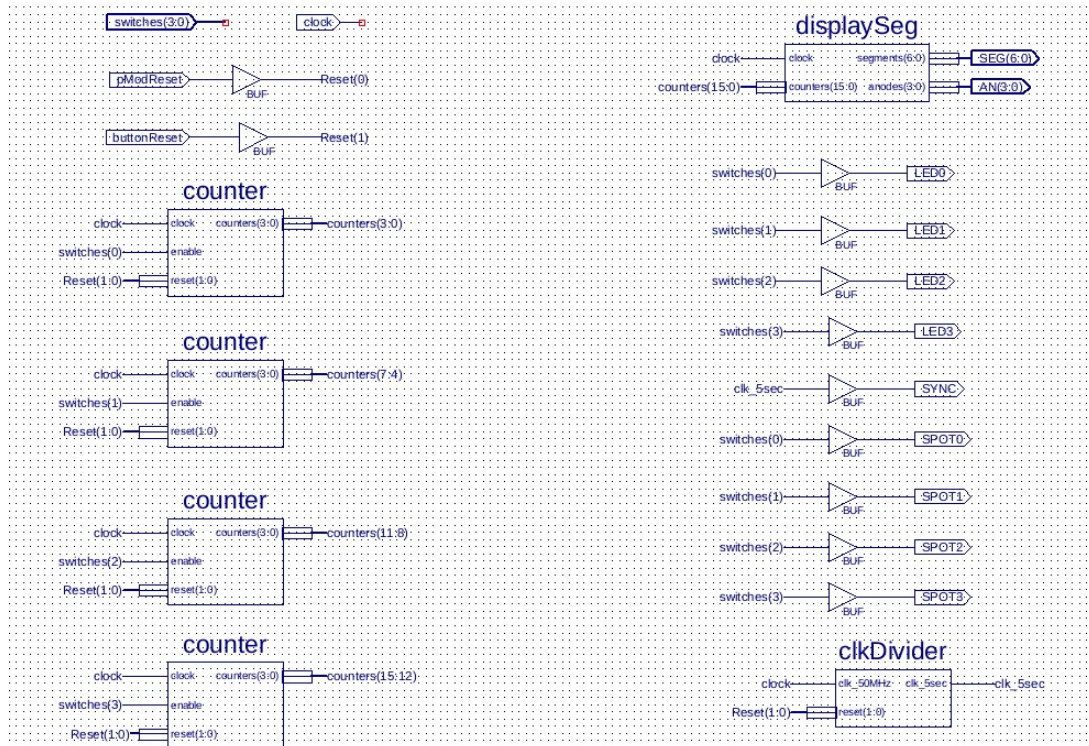


Figura 3 – Arquitetura interna do sistema implementado na Basys2

Como se pode observar na Figura 3, cada switch ativa um contador independente, sendo o valor apresentado nos displays de 7 segmentos. O módulo clkDivider permite gerar um sinal de sincronização mais lento, enquanto o módulo displaySeg controla a apresentação dos valores.

A arquitetura apresentada na Figura 3 representa a implementação do sistema na Basys2, onde cada módulo desempenha uma função específica no controlo dos cacifos, nomeadamente contagem, sincronização e visualização.

4.1.5 Ligações entre o ESP32 e a Basys2

A ligação entre o ESP32 e a Basys2 é realizada através de sinais digitais, utilizando os pinos de entrada/saída (GPIO) do microcontrolador e os conectores Pmod da placa FPGA.

A Basys2 envia para o ESP32 sinais digitais que representam o estado dos lugares do sistema, nomeadamente a ocupação de cada compartimento e a evolução dos contadores. Estes sinais são lidos pelo ESP32 através das suas entradas digitais.

Por outro lado, o ESP32 pode enviar para a Basys2 sinais simples de controlo, como o reset do sistema, permitindo a interação a partir do computador ou da aplicação móvel.

As ligações físicas entre os dois dispositivos são realizadas através de cabos jumper, sendo necessário garantir uma referência comum de massa (GND) para assegurar o correto funcionamento da comunicação digital.

Desta forma, estabelece-se uma comunicação bidirecional simples entre a Basys2 e o ESP32, permitindo a troca de informação entre a lógica digital e o sistema de controlo.

4.1.6 Comunicação com o computador

A comunicação entre o computador e o ESP32 é feita através de porta série via USB, tal como previsto no enunciado para o subsistema equivalente ao Arduino. Esta ligação permite o envio de comandos de controlo e a receção de informação sobre o estado do sistema.

4.1.7 Comunicação com o smartphone

A comunicação com o smartphone é realizada através de Bluetooth, recorrendo ao módulo integrado no ESP32. Esta solução evita a utilização de hardware adicional e permite estabelecer uma ligação sem fios entre o sistema e a aplicação móvel, mantendo o princípio de comunicação série virtual.

4.1.8 Processo de comunicação de dados

O processo de comunicação de dados no sistema EasyCharge descreve o percurso completo da informação desde a geração dos sinais na Basys2 até à sua apresentação nas aplicações de utilizador.

Inicialmente, a Basys2 gera sinais digitais que representam o estado dos lugares do sistema, nomeadamente a ocupação (livre ou ocupado) e os valores dos contadores associados ao tempo de utilização. Estes sinais são enviados para o ESP32 através de ligações digitais entre as duas placas.

De seguida, o ESP32 recebe estes sinais através das suas entradas GPIO e realiza a sua interpretação. A informação é então convertida em mensagens compreensíveis para os sistemas externos, permitindo representar o estado do sistema de forma estruturada.

Após o processamento, o ESP32 envia os dados para o computador através de comunicação série (USB) e para a aplicação móvel através de Bluetooth. Desta forma, o utilizador pode acompanhar o estado dos cacifos em tempo real através das diferentes interfaces.

Assim, o fluxo de comunicação do sistema pode ser representado da seguinte forma:

Basys2 → ESP32 → Computador / Aplicação móvel

Este processo garante a integração entre a lógica digital implementada em hardware e os sistemas de supervisão e controlo, permitindo uma monitorização contínua e eficiente do sistema EasyCharge.

4.1.9 Componentes auxiliares

Para além do ESP32 e da Basys2, foram utilizados os seguintes componentes de apoio:

- Breadboard para prototipagem;
- Cabos jumper para interligação entre o ESP32 e a Basys2;

- Cabos USB para alimentação, programação e comunicação com o computador;

A utilização da combinação ESP32 + Basys2 permitiu implementar a arquitetura funcional do projeto, assegurando a comunicação com o computador, a comunicação com o smartphone e a troca de sinais digitais entre a Basys2 e o ESP32, bem como a simulação do funcionamento do sistema através de lógica implementada em FPGA.

4.2 Circuito

A implementação prática do circuito do sistema EasyCharge baseou-se na interligação entre a placa Basys2, o ESP32, o computador e o smartphone, como está representado na Figura 4. O computador foi utilizado como interface local e como meio de comunicação série com o ESP32, através de ligação USB. O smartphone foi utilizado como interface remota, comunicando com o sistema por Bluetooth. A Basys2 foi responsável pela simulação do comportamento dos lugares de carregamento, enquanto o ESP32 funcionou como elemento intermédio entre a lógica digital implementada na placa e as aplicações de utilizador.

A ligação física entre o ESP32 e a Basys2 foi efetuada através de sinais digitais, recorrendo aos conectores Pmod da placa, tal como indicado no documento de apoio da componente de Sistemas Lógicos. Por esta via, a Basys2 fornece ao ESP32 sinais relativos ao estado dos lugares e à evolução do processo de contagem, enquanto o ESP32 pode enviar para a placa sinais simples de controlo, como o reset acionado a partir das aplicações. Para garantir o correto funcionamento da comunicação digital, foi assegurada uma referência comum de massa entre os dois módulos.

Do ponto de vista físico, a montagem recorreu a cabos USB para alimentação, programação e comunicação com o computador, bem como a cabos jumper para a interligação dos sinais entre placas. Embora exista interligação física real entre os vários dispositivos, o funcionamento do equipamento controlado é simulado na Basys2, o que significa que a ocupação dos lugares, o carregamento e o reset são representados por elementos digitais da própria placa e não por sensores ou atuadores reais. Desta forma, o circuito implementado combina comunicação física real entre módulos com simulação digital do sistema controlado.

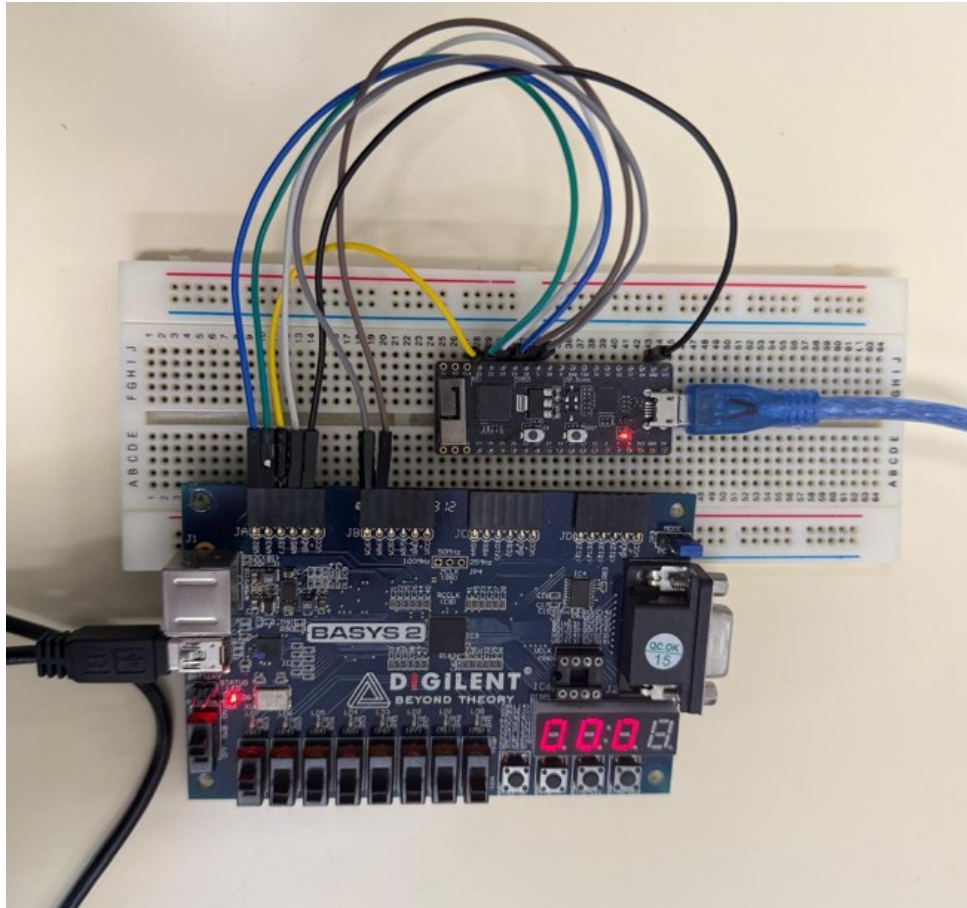


Figura 4 – Montagem prática do sistema EasyCharge.

4.3 Firmware (leitura de sinais e envio de dados)

O firmware desenvolvido para o sistema EasyCharge, executado no ESP32 e responsável pela leitura dos sinais provenientes da Basys2, pela interpretação dos estados do sistema e pelo envio da informação para os meios de monitorização externos.

O firmware constitui um elemento central na ligação entre a lógica implementada em hardware e as aplicações de controlo e supervisão, assegurando o tratamento dos dados recebidos e a sua transmissão de forma adequada.

O ESP32 realiza a leitura de sinais digitais que representam o estado dos lugares (ocupação e contadores), convertendo essa informação em mensagens compreensíveis para o computador e para a aplicação móvel. Estas mensagens são enviadas através de comunicação série (USB) e Bluetooth, permitindo acompanhar o estado do sistema em tempo real.

Desta forma, o firmware garante a integração entre a componente física simulada na Basys2 e as interfaces de utilizador, assegurando a correta transmissão e atualização da informação.

4.3.1 Funcionamento do Firmware

O firmware do sistema EasyCharge é executado no ESP32, sendo responsável por comunicar com a Basys2, interpretar os sinais recebidos e enviar os dados para o sistema de monitorização (computador ou aplicação móvel). Este firmware realiza principalmente duas tarefas: leitura dos estados do sistema e envio de dados.

O ESP32 recebe sinais digitais provenientes da Basys2, que implementa a máquina de estados do sistema. Esses sinais representam o estado atual dos cacifos, nomeadamente a ocupação de cada lugar e o valor dos contadores associados ao tempo de utilização. O microcontrolador lê esses sinais através das suas entradas GPIO e converte-os em informação que pode ser enviada para os sistemas externos.

Posteriormente, o ESP32 envia esses dados através de comunicação série (USB) para o computador e através de Bluetooth para a aplicação móvel, permitindo a monitorização do estado do sistema em tempo real.

4.3.2 Possíveis extensões do sistema

Embora o sistema desenvolvido seja baseado na simulação digital implementada na Basys2, numa versão real do projeto poderiam ser integrados sensores físicos para melhorar o controlo e a segurança dos cacifos.

Por exemplo, poderia ser utilizado um sensor de presença (infravermelho ou ultrassónico) para detetar a existência de uma trotinete no interior do compartimento. Adicionalmente, sensores de posição da porta (como interruptores magnéticos) permitiriam garantir que o carregamento apenas ocorre com a porta fechada.

Poderiam ainda ser utilizados sensores de corrente ou tensão para monitorizar o processo de carregamento, permitindo identificar o estado da bateria.

Estes sensores poderiam ser ligados diretamente ao ESP32, que realizaria a sua leitura e integração com o restante sistema.

4.3.3 Envio de Dados

O envio de dados no sistema EasyCharge ocorre através da leitura, pelo ESP32, dos sinais digitais gerados pela Basys2, seguida da transmissão dessas informações para o computador e para a aplicação móvel.

4.3.3.1 Comunicação Basys2 → ESP32

A Basys2 envia sinais digitais para o ESP32 através de pinos GPIO. Estes sinais representam o estado dos lugares do sistema, nomeadamente a ocupação de cada compartimento e a evolução dos contadores associados ao tempo de utilização.

Os sinais são transmitidos sob a forma de níveis lógicos digitais (0 ou 1), permitindo ao ESP32 interpretar o estado atual do sistema e convertê-lo em mensagens adequadas para envio externo.

Por exemplo, o ESP32 pode converter estes sinais em mensagens como S3:OCUPADO, S2:LIVRE ou S3:1:5 S2:0:0, facilitando a apresentação da informação no computador e na aplicação móvel.

4.3.3.2 Tipo de Dados

Os dados enviados da Basys2 para o ESP32 são essencialmente sinais digitais binários, representando os estados lógicos do sistema.

Por exemplo:

Sinal	Valor	Significado
S3	1	Lugar 3 ocupado
S3	0	Lugar 3 livre
S2	1	Lugar 2 ocupado
S2	0	Lugar 2 livre

Tabela 3 – Exemplo de interpretação dos sinais enviados da Basys2 para o ESP32

O firmware do ESP32 lê estes valores através das entradas GPIO e converte-os em mensagens de estado, que são posteriormente enviadas para o computador e para a aplicação móvel.

4.3.3.3 Envio para o sistema

Depois de interpretar os sinais recebidos da Basys2, o ESP32 envia os dados para o sistema de monitorização através de comunicação série (USB) para o computador e através de Bluetooth para a aplicação móvel.

As mensagens enviadas incluem informações relativas ao estado dos cacifos, nomeadamente a ocupação de cada lugar e os valores dos contadores associados ao tempo de utilização.

Desta forma, o firmware permite que o estado do sistema EasyCharge seja monitorizado em tempo real através das diferentes interfaces de utilizador.

4.3.4 Código do ESP32

O código implementado no ESP32 tem como principal objetivo realizar a leitura dos sinais digitais provenientes da placa Basys2, processar essa informação e disponibilizá-la ao utilizador através de comunicação série e Bluetooth. Para além disso, este módulo gere o estado de carregamento de cada cacifo, permitindo iniciar, parar e monitorizar o processo de carga de forma individual.

Numa primeira fase, no método `setup()`, são configurados os pinos de entrada associados aos sinais enviados pela Basys2, correspondentes ao estado dos quatro lugares do sistema e ao sinal de sincronização (SYNC). É também inicializada a comunicação série com o computador e a comunicação Bluetooth com a aplicação móvel.

O excerto seguinte mostra a configuração inicial do sistema:

```
void setup() {  
  Serial.begin(9600);  
  pinMode(23, INPUT);  
  pinMode(22, INPUT);  
  pinMode(21, INPUT);  
  pinMode(19, INPUT);  
  pinMode(18, INPUT);  
  pinMode(5, INPUT);  
  pinMode(2, OUTPUT);  
  SerialBT.begin("EasyCharge");  
}
```

Durante a execução contínua do método `loop()`, o ESP32 realiza a leitura dos sinais digitais provenientes da Basys2 através dos pinos GPIO, correspondentes ao estado dos diferentes lugares do sistema:

```
bool s3 = digitalRead(23);  
bool s2 = digitalRead(19);  
bool s1 = digitalRead(21);  
bool s0 = digitalRead(22);
```

Sempre que é detetada uma alteração no estado de um dos lugares, o sistema envia automaticamente uma mensagem via Bluetooth, permitindo à aplicação móvel atualizar a informação apresentada ao utilizador em tempo real. Caso um lugar deixe de estar ocupado enquanto a carga se encontra ativa, o sistema termina automaticamente o processo de carregamento desse cacifo e reinicia o respetivo contador.

```
if (s3 != prev3) {  
  SerialBT.println(s3 ? "S3:OCUPADO" : "S3:LIVRE");  
  if (!s3 && cargaAtiva[3]) {  
    cargaAtiva[3] = false;  
    counter3 = 0;  
    SerialBT.println("CARGA:TERMINADA:3");  
  }  
  prev3 = s3;  
}
```

O firmware implementa ainda uma função responsável por enviar o estado completo do sistema sempre que é estabelecida uma nova ligação Bluetooth ou quando esse estado é solicitado pela aplicação. Esta função envia a ocupação de cada cacifo e o estado da carga associada.

```
void enviarEstadoCompleto() {  
    SerialBT.println(s3 ? "S3:OCUPADO" : "S3:LIVRE");  
    ...  
    SerialBT.println(cargaAtiva[3] ? "CARGA:ATIVA:3" : "CARGA:INATIVA:3");  
}
```

Para além da monitorização automática, o ESP32 também recebe comandos enviados pela aplicação móvel através de Bluetooth. Entre esses comandos encontram-se o início e paragem da carga, a abertura e fecho da porta e o pedido do estado completo do sistema.

```
if (comando.startsWith("CARGA:INICIAR:")) {  
    int cacifo = comando.substring(14).toInt();  
    cargaAtiva[cacifo] = true;  
    SerialBT.println("CARGA:INICIADA:" + String(cacifo));  
}
```

O sistema responde igualmente a comandos recebidos do computador através da porta série, nomeadamente ESTADO e RESET, permitindo a supervisão local do funcionamento do sistema.

O sinal de sincronização (SYNC), proveniente da Basys2, é utilizado como referência temporal para o incremento dos contadores. No entanto, a contagem só é realizada quando o respetivo lugar se encontra ocupado e quando a carga foi autorizada para esse cacifo.

```
if (digitalRead(18)) {  
    if (s3 && cargaAtiva[3]) counter3++;  
    if (s2 && cargaAtiva[2]) counter2++;  
    if (s1 && cargaAtiva[1]) counter1++;  
    if (s0 && cargaAtiva[0]) counter0++;  
}
```

A comunicação entre o ESP32 e a aplicação móvel baseia-se num protocolo textual simples, em que cada mensagem segue o formato CATEGORIA:ACAO:PARAMETRO. Este formato permite representar, de forma estruturada, tanto comandos enviados pela aplicação como estados e eventos devolvidos pelo microcontrolador. Entre os comandos recebidos pelo ESP32 incluem-se, por exemplo, CARGA:INICIAR:X, CARGA:PARAR:X, PORTA:ABRIR:X, PORTA:FECHAR:X e ESTADO. Por sua vez, o ESP32 envia para a aplicação mensagens como S0:OCUPADO, S1:LIVRE, CARGA:INICIADA:X, CARGA:TERMINADA:X, PORTA:ABERTA:X e PORTA:FECHADA:X. Desta forma, a aplicação consegue interpretar facilmente o estado dos cacifos e reagir aos eventos ocorridos no sistema.

Desta forma, o ESP32 funciona como elemento intermédio do sistema, responsável por interpretar os sinais provenientes da Basys2, gerir a lógica de carregamento autorizada, atualizar os contadores e assegurar a comunicação entre o hardware e as interfaces de utilizador.

4.3.5 Código da Basys2

A implementação da lógica digital na Basys2 foi realizada no ambiente ISE Xilinx, recorrendo a módulos em Verilog e a um ficheiro de restrições .ucf, responsável pela associação entre os sinais lógicos do projeto e os pinos físicos da placa.

4.3.5.1 Ficheiro de implementação .ucf

O ficheiro `implementation.ucf` define a correspondência entre os sinais internos do sistema e os componentes físicos disponíveis na Basys2, como os displays de 7 segmentos, LEDs, botões, switches e conectores Pmod.

```
// Display
NET "SEG<0>" LOC = "L14 ";
NET "SEG<1>" LOC = "H12 ";
NET "SEG<2>" LOC = "N14 ";
NET "SEG<3>" LOC = "N11 ";
NET "SEG<4>" LOC = "P12 ";
NET "SEG<5>" LOC = "L13 ";
NET "SEG<6>" LOC = "M12 ";
NET "AN<0>" LOC = "K14 ";
NET "AN<1>" LOC = "M13 ";
NET "AN<2>" LOC = "J12 ";
NET "AN<3>" LOC = "F12 ";

// Clock
NET "clock" LOC = "B8 ";

// LEDs
NET "LED3" LOC = "P6 ";
NET "LED2" LOC = "M5 ";
NET "LED1" LOC = "M11 ";
NET "LED0" LOC = "P7 ";

// Buttons
NET "buttonReset" LOC = "G12 "; //BTN0
```

```
// Switches
NET "switches <3>" LOC = "B4 ";
NET "switches <2>" LOC = "P11 ";
NET "switches <1>" LOC = "L3 ";
NET "switches <0>" LOC = "K3 ";

// Pmod
NET "SYNC" LOC = "B6 ";          // JB1
NET "pModReset" LOC = "C6 ";     // JB2
NET "SPOT3" LOC = "B2 ";         // JA1
NET "SPOT2" LOC = "A3 ";         // JA2
NET "SPOT1" LOC = "J3 ";         // JA3
NET "SPOT0" LOC = "B5 ";         // JA4
```

Neste ficheiro, os sinais SEG e AN são associados aos displays de 7 segmentos, permitindo apresentar os valores dos contadores. O sinal clock corresponde ao relógio principal da placa, utilizado para sincronizar o funcionamento dos módulos internos.

Os sinais LED0 a LED3 são utilizados para indicar visualmente o estado de cada lugar, enquanto o sinal buttonReset corresponde ao botão físico de reset da Basys2.

Os sinais switches<3:0> representam os quatro switches utilizados para simular a ocupação dos compartimentos. Já os sinais definidos nos conectores Pmod permitem a comunicação com o ESP32: o sinal SYNC é utilizado para sincronização, o sinal pModReset permite receber o reset externo e os sinais SPOT0 a SPOT3 representam a informação relativa ao estado dos diferentes lugares.

Desta forma, o ficheiro .ucf assegura a correta ligação entre a lógica descrita no projeto e os recursos físicos da placa Basys2, sendo essencial para o funcionamento global do sistema EasyCharge.

5 Aplicação e Deployment

Neste capítulo são apresentadas as componentes da solução responsáveis pela interação com o utilizador, pelo armazenamento de informação e pela disponibilização dos dados para monitorização e controlo do sistema. Numa primeira fase, é abordada a base de dados, utilizada para registar e organizar a informação gerada durante o funcionamento do sistema. De seguida, é apresentada a dashboard ou aplicação desenvolvida, através da qual o utilizador pode acompanhar o estado do sistema e interagir com os diferentes elementos disponíveis. Posteriormente, são descritas as funcionalidades de visualização e controlo, essenciais para a supervisão do equipamento em tempo real. Por fim, são referidas algumas funcionalidades complementares da aplicação, que contribuem para tornar a solução mais completa do ponto de vista da experiência de utilização e da gestão da conta do utilizador.

5.1 Base de Dados

A aplicação EasyCharge utiliza duas soluções complementares de armazenamento de dados: o *CloudDB* do MIT App Inventor e o *TinyDB* local do dispositivo.

5.1.1 CloudDB

O *CloudDB* é um sistema de armazenamento em nuvem disponibilizado pelo MIT App Inventor, que permite guardar e sincronizar dados online entre diferentes dispositivos. Os dados são organizados em pares chave-valor, sendo utilizados no projeto para armazenar informação persistente relativa aos utilizadores, ao histórico de carregamentos e aos pontos de carga.

A estrutura de dados definida no *CloudDB* é a seguinte:

```
utilizadores/  
  email@exemplo.com/  
    nome: "Rodrigo Belchior"  
    password: "*****"  
    telemovel: "912345678"  
  
historico/  
  email@exemplo.com/  
    lista de carregamentos:  
      [  
        "2026-04-12 10:55|IPS Tecnologia|1|1.40|0.18|00:03:19",  
        "2026-04-12 15:41|IPS Tecnologia|2|0.50|0.06|00:01:39"
```



```
]
```

```
pontos/
```

```
  lista de pontos de carga:
```

```
  [
    "IPS Tecnologia|Disponível|4|38.521838|-8.838941",
    "IPS Educação|Disponível|4|38.519644|-8.838727"
  ]
```

Cada registo do histórico contém seis campos separados pelo símbolo |: data e hora, local, número do cacifo, energia consumida, custo e duração. Cada ponto de carga contém cinco campos: nome do local, estado, número de cacifos, latitude e longitude.

5.1.2 TinyDB

O *TinyDB* é uma base de dados local que armazena informação diretamente no dispositivo do utilizador. Os dados persistem entre sessões, sendo utilizados para guardar informação de sessão e estatísticas locais da aplicação.

A estrutura usada no *TinyDB* é a seguinte:

```
email_sessao    -> rodrigo@ips.pt
total_cargas    -> 5
total_kwh       -> 7.35
total_custo     -> 0.95
ultima_carga    -> Cacifo 1 · 1.40 kWh · 0.18€
```

5.1.3 Fluxo de Dados

No processo de registo e autenticação, a aplicação guarda os dados do utilizador no *CloudDB* e regista o email da sessão no *TinyDB*. Durante o carregamento, a aplicação comunica com o ESP32 via Bluetooth, calcula os valores associados ao processo e atualiza o histórico no *CloudDB*, bem como os totais locais no *TinyDB*. Na visualização do histórico, a aplicação lê os totais armazenados localmente e consulta a lista de carregamentos guardada na cloud.

5.1.4 Segurança

No contexto deste protótipo académico, os dados dos utilizadores são identificados através do respetivo email, utilizado como chave de organização da informação. No entanto, as palavras-passe foram armazenadas de forma simples, o que não é adequado para uma aplicação real. Num sistema de produção, seria necessário aplicar mecanismos de proteção, como cifragem ou *hashing*, por exemplo com SHA-256.

5.1.5 Limitações

O *CloudDB* do MIT App Inventor constitui uma solução adequada para fins acadêmicos e de prototipagem, mas apresenta limitações para utilização em produção. Entre essas limitações destacam-se a ausência de mecanismos avançados de segurança, a dependência da infraestrutura do MIT e a reduzida escalabilidade. Numa versão futura ou comercial do sistema, seria recomendável migrar para uma solução mais robusta, como Firebase, Supabase ou outro serviço profissional de base de dados.

5.2 Dashboard ou App

A página principal da aplicação móvel (Figura 5) funciona como dashboard geral do sistema, apresentando ao utilizador uma visão resumida do estado atual da sua utilização e das principais funcionalidades disponíveis. Nesta página são mostrados elementos como a identificação do utilizador, o estado corrente do sistema, informação relativa à última carga efetuada e a indicação de disponibilidade.

Para facilitar a navegação, a dashboard inclui atalhos diretos para as funcionalidades principais da aplicação, nomeadamente o início de um carregamento, a consulta do mapa de carregadores, o acesso ao histórico de cargas e a gestão do perfil do utilizador. Adicionalmente, esta página apresenta um resumo mensal com indicadores relevantes, como o número de cargas efetuadas, a energia carregada em kWh e o gasto associado em euros.

A dashboard inclui ainda uma barra de navegação inferior, que permite ao utilizador alternar rapidamente entre as áreas principais da aplicação, tornando a interação mais simples, intuitiva e adequada a um contexto de utilização móvel.

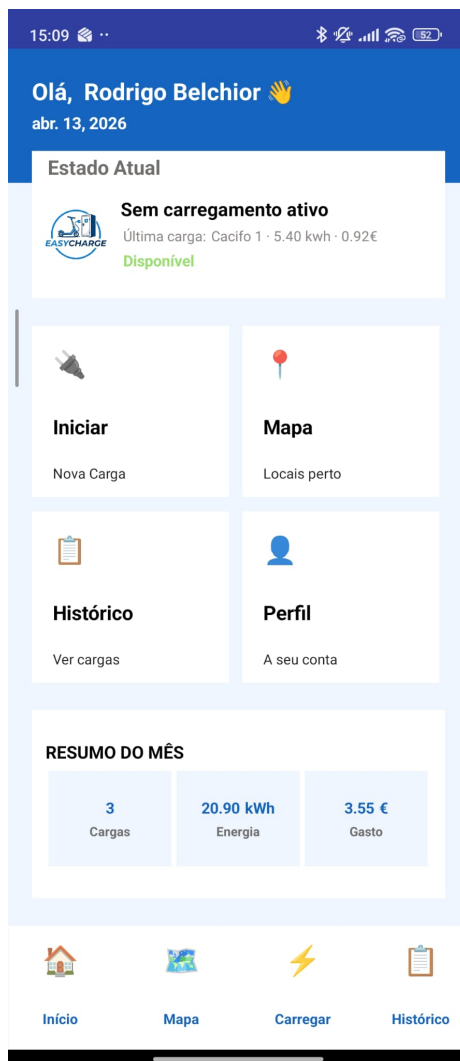


Figura 5 – Página Principal

A interface da aplicação foi desenvolvida com preocupação ao nível da usabilidade, procurando apresentar informação clara, navegação simples e acesso rápido às funcionalidades principais. A dashboard fornece ao utilizador uma visão imediata do estado atual do sistema, enquanto a organização por páginas e a barra de navegação inferior facilitam o acesso ao mapa, ao carregamento, ao histórico e ao perfil. Para além disso, a aplicação utiliza linguagem simples, indicadores visuais de estado e ações bem identificadas, contribuindo para uma utilização mais intuitiva e eficiente.

5.3 Visualização e Conteúdo

A interface gráfica desenvolvida para o sistema de cacifos foi organizada em várias páginas, com o objetivo de facilitar a monitorização, o controlo e a configuração do sistema. A estrutura adotada permite ao utilizador aceder rapidamente às principais funcionalidades, mantendo uma apresentação clara, intuitiva e visualmente organizada. As páginas principais da aplicação são a

página de carregamento, a página de histórico, a página do mapa e a página do perfil.

5.3.1 **Página de Carregamento**

A página de carregamento, Figura 6, constitui o principal ponto de interação direta do utilizador com o sistema EasyCharge. Nesta área da aplicação, o utilizador pode seleccionar um dos cacifos disponíveis e consultar, de forma detalhada, o respetivo estado atual. Entre as informações apresentadas encontram-se o estado do cacifo, o estado da porta, a deteção da trotinete no interior do compartimento, o tempo de carga decorrido e o nível de bateria associado ao processo de carregamento.

Para além da componente informativa, esta página apresenta também o estado da ligação Bluetooth ao sistema, permitindo confirmar a comunicação com o dispositivo físico. A interface integra ainda as ações de controlo mais relevantes, nomeadamente o início do carregamento, bem como os comandos de abertura e fecho da porta do cacifo selecionado. Adicionalmente, é apresentado o último evento registado, contribuindo para uma monitorização mais clara e imediata do sistema.

A atualização desta página é realizada com base nas mensagens recebidas por Bluetooth a partir do ESP32. Sempre que ocorre uma alteração de estado no sistema, como ocupação do cacifo, abertura ou fecho da porta, início ou fim do carregamento, a aplicação interpreta a mensagem recebida e atualiza imediatamente os elementos visuais apresentados ao utilizador.

Desta forma, a página de carregamento concentra as funcionalidades essenciais de operação do sistema, permitindo acompanhar e controlar o processo de forma simples, direta e intuitiva.



Figura 6 – Página de Carregamentos

5.3.2 Página de Histórico

A página de histórico, representada na Figura 7, permite ao utilizador consultar o registo das cargas realizadas e acompanhar a sua atividade ao longo do tempo. Nesta área da aplicação são apresentados dados resumidos sobre a utilização do sistema, incluindo o número de cargas efetuadas, a energia total consumida e o valor gasto no período considerado.

Para além da componente estatística, esta página apresenta também uma listagem cronológica das sessões de carregamento registadas. Em cada registo podem ser consultadas informações como o local utilizado, o cacifo associado, a data e hora da sessão, a energia carregada, a duração da utilização e o custo correspondente.

Desta forma, a página de histórico contribui para uma utilização mais informada do sistema, permitindo ao utilizador acompanhar os seus hábitos de carregamento e consultar rapidamente o registo detalhado das operações efetuadas.



Figura 7 – Histórico de cargas

5.3.3 Página do Mapa

A página de mapa (Figura 8) permite ao utilizador visualizar geograficamente os pontos de carregamento disponíveis no sistema. Esta funcionalidade facilita a localização de estações próximas e fornece uma perspetiva espacial da rede de carregadores, tornando a aplicação mais próxima de um cenário real de utilização.

Para além da representação em mapa, esta página apresenta também uma lista resumida de pontos de carregamento próximos, indicando o nome do local e a respetiva disponibilidade. Desta forma, o utilizador pode identificar rapidamente os locais mais convenientes para utilização do sistema, tanto através da visualização geográfica como da listagem textual.

A integração desta funcionalidade contribui para melhorar a experiência de utilização da aplicação, uma vez que permite não apenas controlar e consultar carregamentos já efetuados,

mas também localizar previamente pontos disponíveis para futuras utilizações. Para além da representação em mapa com marcadores de localização, esta página apresenta também uma lista resumida dos pontos de carregamento disponíveis, indicando o nome do local, o estado de disponibilidade e o número de cacifos existentes.



Figura 8 – Página de mapa de carregadores

5.3.4 Página de Perfil

A página de perfil, Figura 9, reúne a informação associada à conta do utilizador e disponibiliza acesso a um conjunto de funcionalidades complementares da aplicação. Nesta área é possível consultar a identificação do utilizador, o respetivo email, o estado da conta e dados resumidos de utilização, como o número total de cargas efetuadas, a energia acumulada em kWh e o valor total gasto.

Para além dessa informação, a página de perfil funciona também como ponto de acesso a várias opções de gestão e suporte. Entre essas funcionalidades incluem-se a consulta e edição de dados pessoais, a gestão de métodos de pagamento, a consulta de ajuda e perguntas frequentes,

bem como a possibilidade de contactar o suporte da aplicação. A página inclui ainda a opção de terminar sessão, assegurando o controlo do acesso por parte do utilizador.

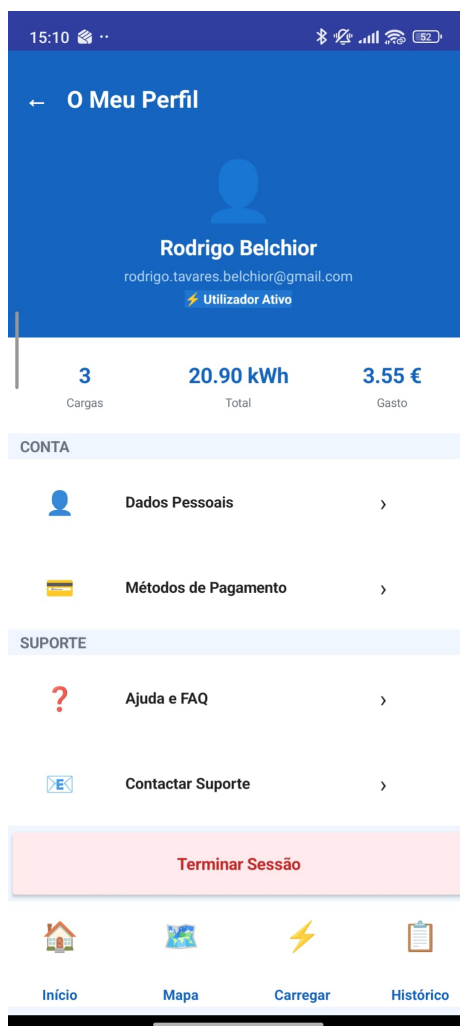


Figura 9 – Página de perfil

5.3.5 Funcionalidades Complementares

Para além das páginas principais de navegação e controlo, a aplicação móvel integra ainda um conjunto de funcionalidades complementares que tornam a solução mais completa e mais próxima de uma aplicação real de gestão de carregamento. Estas funcionalidades não se limitam ao controlo direto dos cacifos, abrangendo também aspetos de autenticação, gestão da conta e apoio ao utilizador.

Entre essas funcionalidades encontram-se as páginas de autenticação e criação de conta, que permitem efetuar o registo e o acesso seguro à aplicação. Estão também disponíveis páginas dedicadas aos dados pessoais do utilizador, à gestão de métodos de pagamento e à consulta de notificações, permitindo um maior controlo sobre a utilização do sistema e sobre a informação associada à conta.

Adicionalmente, a aplicação inclui secções de ajuda e perguntas frequentes, bem como uma área de contacto com o suporte, com o objetivo de facilitar a resolução de dúvidas e melhorar a experiência de utilização. No seu conjunto, estas funcionalidades complementares reforçam a componente prática e funcional da aplicação desenvolvida, demonstrando uma preocupação não apenas com o controlo técnico do sistema, mas também com a experiência global do utilizador.

6 Gestão de Código no Bitbucket

Para o desenvolvimento do projeto EasyCharge foi utilizada a plataforma Bitbucket, que permite realizar o controlo de versões do código e facilitar o trabalho em equipa. Esta plataforma possibilita que vários membros do grupo trabalhem no mesmo projeto, mantendo um histórico de todas as alterações realizadas.

O primeiro passo consistiu na criação de um repositório no Bitbucket, onde foi armazenado todo o código e documentação do projeto. Após a criação do repositório, cada elemento do grupo teve acesso ao mesmo, permitindo colaborar no desenvolvimento.

De seguida, o repositório foi clonado para os computadores dos membros da equipa utilizando o sistema de controlo de versões Git. Este processo permitiu criar uma cópia local do projeto, onde cada elemento pôde desenvolver e modificar o código.

Durante o desenvolvimento, sempre que foram realizadas alterações ao projeto, foi utilizado o seguinte procedimento:

1. Primeiro foram feitas as alterações no código local.
2. Depois as alterações foram adicionadas ao sistema de controlo de versões através do comando `git add`.
3. De seguida foi criado um registo das alterações com o comando `git commit`, acompanhado de uma mensagem a descrever as modificações realizadas.
4. Finalmente, as alterações foram enviadas para o repositório remoto no Bitbucket utilizando o comando `git push`.

Este processo permitiu manter todas as versões do projeto organizadas e acessíveis, garantindo que o trabalho realizado por cada membro da equipa ficava registado.

No contexto deste projeto, foi escolhido o Bitbucket devido à sua facilidade de utilização e às funcionalidades de colaboração e gestão de versões disponibilizadas.

6.1 Comparação entre Bitbucket, GitHub e GitLab

Existem várias plataformas que permitem realizar controlo de versões e gestão de projetos de software, sendo as mais conhecidas o Bitbucket, GitHub e GitLab.

O Bitbucket é uma plataforma muito utilizada em ambientes académicos e empresariais, especialmente quando integrada com outras ferramentas da Atlassian, como o Jira. Esta integração facilita a gestão de tarefas e o acompanhamento do desenvolvimento do projeto.

O GitHub é atualmente a plataforma mais popular para alojamento de repositórios Git. É amplamente utilizada por programadores em todo o mundo e possui uma grande comunidade, o que facilita a partilha de projetos e a colaboração entre programadores.

Por outro lado, o GitLab oferece funcionalidades semelhantes às outras plataformas, mas destaca-se por integrar várias ferramentas de desenvolvimento diretamente na plataforma.

De forma geral, todas estas plataformas permitem realizar controlo de versões e colaboração em projetos de software. No entanto, no contexto deste projeto foi escolhido o Bitbucket, principalmente pela sua integração com outras ferramentas utilizadas no desenvolvimento.

7 Análise dos Sprints

Ao longo do desenvolvimento do projeto, foi efetuado o acompanhamento da evolução dos diferentes sprints através dos gráficos burndown disponibilizados pelo Jira. Estes gráficos permitiram comparar o progresso real do trabalho com o ritmo de execução inicialmente previsto, facilitando a identificação de desvios ao planeamento e a análise do desempenho da equipa em cada fase do projeto.

No caso do sprint correspondente à fase de Testes, Resultados e Conclusões, observa-se que a evolução do trabalho decorreu de forma globalmente positiva. Apesar de a redução do número de tarefas ter ocorrido mais significativamente na fase final do sprint, todas as atividades previstas foram concluídas dentro do período estabelecido. O gráfico demonstra que o trabalho foi finalizado atempadamente, permitindo considerar este sprint como um exemplo de execução bem-sucedida, com cumprimento dos objetivos definidos.

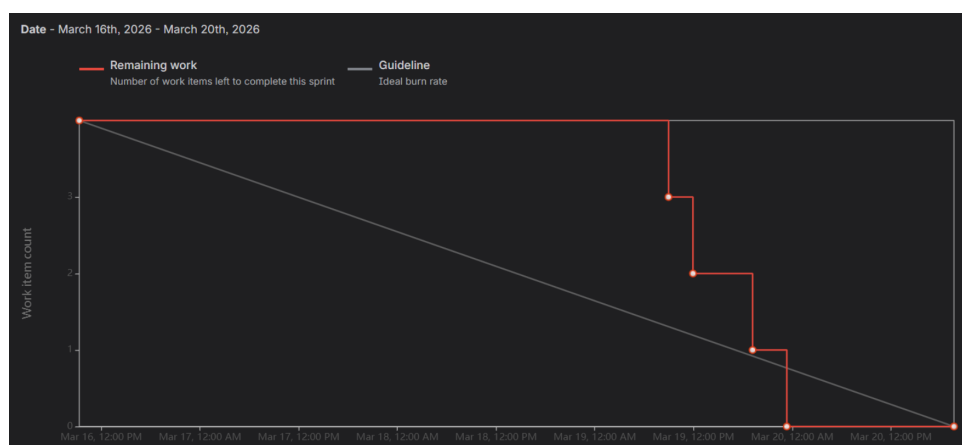


Figura 10 – Sprint Testes, Resultados e Conclusões

Por outro lado, o sprint relativo à fase de Análise e Planeamento revelou maiores dificuldades no cumprimento do planeamento inicial. Conforme se observa no respetivo gráfico burndown, o número de tarefas manteve-se inalterado durante grande parte do período previsto, verificando-se apenas uma redução mais acentuada próximo da data de conclusão. Este comportamento indica uma concentração do trabalho na fase final do sprint, o que dificultou o acompanhamento da linha ideal de progresso e traduziu uma gestão temporal menos equilibrada. Embora as tarefas tenham sido concluídas, o sprint não decorreu de forma tão eficiente como o anterior.

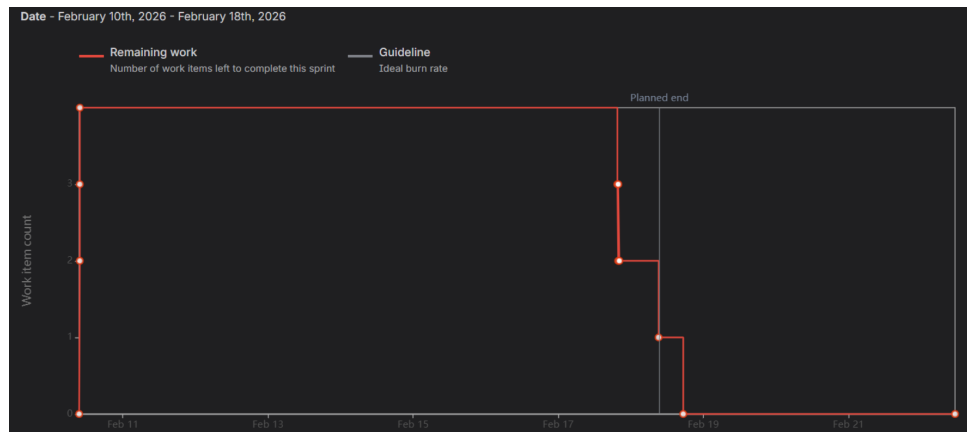


Figura 11 – Sprint Análise e Planejamento

A análise destes dois casos evidencia a importância do planejamento contínuo e da distribuição equilibrada das tarefas ao longo de cada sprint. Enquanto o primeiro exemplo demonstra um desempenho mais eficaz e alinhado com os objetivos temporais definidos, o segundo evidencia a necessidade de melhorar a gestão do trabalho, de modo a evitar acumulações de tarefas nas fases finais e a garantir uma execução mais estável e previsível.

8 Testes, Resultados e Conclusão

Neste capítulo são apresentados os testes realizados ao sistema EasyCharge, bem como os principais resultados obtidos durante a sua implementação e validação.

Numa primeira fase, são descritos os testes efetuados, com o objetivo de verificar o correto funcionamento das diferentes componentes do sistema e a integração entre hardware e software. De seguida, é apresentada uma proposta de manutenção, tendo em vista a continuidade do funcionamento adequado da solução ao longo do tempo.

Posteriormente, são analisadas as principais limitações identificadas no desenvolvimento do projeto, evidenciando aspetos que condicionam o desempenho ou a abrangência da solução. Por fim, são apresentadas possíveis melhorias futuras, que poderão contribuir para a evolução e otimização do sistema desenvolvido.

8.1 Testes e Resultados Obtidos

Para validar o sistema desenvolvido, foram realizados diversos testes práticos de hardware e software com o objetivo de verificar o correto funcionamento do sistema.

Estes testes incluíram a verificação da leitura dos sinais provenientes da Basys2, o funcionamento dos contadores, a resposta ao comando de reset e a comunicação entre o ESP32, o computador e a aplicação móvel.

8.1.1 Comunicação

Foi testada a comunicação entre a aplicação e o ESP32, bem como entre o ESP32 e a Basys2. Para isso, foram enviados comandos simples, como abrir um cacifo, fechar um cacifo ou consultar o estado do sistema. O teste foi considerado válido quando o comando chegou corretamente ao destino, a resposta devolvida correspondeu ao pedido efetuado e o estado apresentado ao utilizador foi coerente com o estado real do sistema. Uma vez que o microcontrolador é responsável por gerar sinais de controlo, receber estados e trocar mensagens com as interfaces de utilizador, esta validação revelou-se essencial para garantir o correto funcionamento do sistema.

8.1.2 Abertura e Fecho dos Cacifos

Foram realizados testes ao funcionamento dos quatro lugares do sistema, simulando a ocupação e desocupação dos cacifos através dos switches da Basys2. O procedimento consistiu em ativar e desativar os switches correspondentes a cada lugar, verificando se o sistema atualizava corretamente o estado do cacifo. Foi também analisada a resposta do sistema ao nível dos LEDs, dos displays de 7 segmentos e das mensagens enviadas para o ESP32 e para a aplicação. O teste

foi considerado bem sucedido quando o estado apresentado na Basys2, no ESP32 e na aplicação era coerente, não existindo inconsistências entre a simulação local e a informação apresentada ao utilizador.

8.1.3 Ocupação e disponibilidade

Foram realizados testes para verificar se o sistema distinguia corretamente lugares livres e ocupados. Para tal, foram simuladas diferentes combinações de utilização dos quatro lugares através dos switches da Basys2. O teste foi considerado bem sucedido quando o estado apresentado nos LEDs, nos displays, no ESP32 e na aplicação coincidiu com a ocupação simulada.

8.1.4 Interface com o utilizador

Foi testada a interface com o utilizador, no computador e na aplicação móvel, com o objetivo de verificar se o estado dos lugares era apresentado corretamente. O teste foi considerado positivo quando a informação recebida a partir do ESP32 correspondia ao estado real do sistema e era atualizada de forma coerente.

8.2 Propostas de manutenção

Para garantir o bom funcionamento do sistema EasyCharge, é importante definir algumas estratégias de manutenção que permitam evitar falhas e prolongar a vida útil dos equipamentos.

8.2.1 Manutenção Preventiva

Consiste na realização de verificações periódicas ao sistema antes que ocorram problemas. Isto inclui verificar o estado das ligações elétricas, confirmar que os cacifos funcionam corretamente e testar o funcionamento do carregamento simulado. Também é importante verificar o estado do ESP32, da Basys2 e das ligações entre os dispositivos.

8.2.2 Manutenção Corretiva

Este tipo de manutenção ocorre quando é detetada uma falha no sistema. Por exemplo, caso um cacifo deixe de funcionar corretamente, ocorra um erro no sistema ou exista um problema de comunicação entre os dispositivos, deve ser realizada a substituição ou reparação do componente afetado.

8.2.3 Manutenção de Software

Além do hardware, também é necessário manter o software atualizado. Isto inclui corrigir erros no sistema, melhorar o firmware do ESP32, atualizar a aplicação de controlo e garantir que

a comunicação entre os dispositivos continua a funcionar corretamente.

8.2.4 Monitorização do Sistema

O sistema pode registar mensagens de estado ou eventos, permitindo identificar possíveis anomalias no funcionamento. Desta forma, é possível identificar problemas e atuar rapidamente, evitando que afetem o funcionamento do sistema.

8.3 Limitações

O sistema EasyCharge, sendo um protótipo académico, apresenta algumas limitações que podem afetar o seu funcionamento em contexto real.

- **Dependência de ligação física:** A comunicação entre a Basys2 e o ESP32 é feita por ligações diretas (GPIO/UART), o que limita a distância entre os componentes.
- **Comunicação limitada:** O uso de Bluetooth pode ter alcance reduzido e menor fiabilidade em ambientes com interferências.
- **Controlo de carregamento básico:** O sistema não implementa ainda um controlo inteligente da energia, podendo não otimizar o consumo ou proteger totalmente as baterias.
- **Falta de sensores reais:** Como não são utilizados sensores físicos reais, o sistema depende de estados simulados, o que reduz a precisão em situações reais.
- **Segurança simples:** Os mecanismos de autenticação ainda são básicos, podendo não ser suficientes para um sistema real com muitos utilizadores.
- **Escalabilidade reduzida:** O sistema foi pensado para poucos cacifos, sendo difícil de expandir sem alterações na arquitetura.
- **Dependência de ligação ao sistema central:** Caso haja falhas de comunicação com o computador, algumas funcionalidades podem ficar indisponíveis.

8.4 Melhorias Futuras

Para evoluir o sistema EasyCharge, podem ser consideradas várias melhorias que tornariam a solução mais completa e eficiente. No futuro, o sistema poderá incluir controlo inteligente do carregamento, permitindo gerir melhor a energia, evitar sobrecargas e dar prioridade a determinados utilizadores ou níveis de bateria. A aplicação móvel poderá também ser melhorada com funcionalidades como reservas antecipadas de cacifos, histórico de utilizações e notificações em tempo real mais detalhadas. Além disso, o sistema poderá evoluir para uma solução ligada à cloud, permitindo armazenar dados online, aceder remotamente ao estado dos cacifos e

realizar uma gestão centralizada de várias estações. Do ponto de vista da segurança, poderão ser implementados métodos de autenticação mais robustos, como autenticação em dois fatores ou utilização de códigos QR para acesso aos cacifos. Por outro lado, com uma maior recolha de dados, será possível analisar padrões de utilização, identificar horários de maior procura e melhorar a gestão dos recursos disponíveis. Finalmente, o sistema poderá ser adaptado para suportar um maior número de cacifos e locais, permitindo a sua expansão para diferentes cidades e contextos de utilização.

9 Conclusão

Ao longo deste projeto foi desenvolvida a solução EasyCharge, concebida para responder à necessidade de armazenamento e carregamento de trotinetes elétricas, integrando diferentes componentes de hardware e software numa arquitetura funcional e coerente. O sistema proposto permitiu combinar a placa programável Basys2, responsável pela lógica de controlo e simulação do equipamento, com o microcontrolador ESP32, encarregado da comunicação com o computador, com o smartphone e com os restantes módulos do sistema.

O desenvolvimento do EasyCharge permitiu cumprir o objetivo geral definido para o projeto, nomeadamente a criação de um sistema de controlo de equipamento envolvendo hardware programável, microcontrolador, aplicação móvel e comunicação entre módulos. Para além disso, os objetivos específicos foram concretizados através da implementação da máquina de estados, da definição da comunicação entre a Basys2 e o ESP32, e do desenvolvimento das interfaces de monitorização. Desta forma, o projeto constituiu uma aplicação prática dos conhecimentos adquiridos ao longo do curso, em áreas como sistemas digitais, microcontroladores, programação, comunicações e desenvolvimento de aplicações.

A solução desenvolvida demonstrou também a importância de uma arquitetura modular, na qual cada componente desempenha uma função específica e complementar. O ESP32 revelou-se adequado como elemento intermédio de controlo e comunicação, enquanto a Basys2 permitiu implementar a lógica digital do sistema de forma estruturada. A aplicação móvel e a interface no computador permitiram acompanhar o estado do sistema, tornando a solução mais completa e mais próxima de uma aplicação real.

Apesar dos resultados alcançados, o projeto apresenta algumas limitações próprias de um protótipo académico, nomeadamente a utilização de estados simulados em vez de sensores físicos reais, a simplicidade de alguns mecanismos de controlo e a reduzida escalabilidade do sistema para contextos de maior dimensão. Ainda assim, essas limitações não retiram valor ao trabalho desenvolvido; pelo contrário, evidenciam possíveis linhas de evolução futura, como a integração de sensores reais, a melhoria da segurança, a ligação a serviços cloud e a expansão para um maior número de cacifos.

Em conclusão, o projeto EasyCharge permitiu desenvolver uma solução tecnicamente consistente, funcional e adequada aos objetivos propostos, demonstrando a viabilidade de um sistema de cacifos inteligentes para armazenamento e carregamento simulado de trotinetes elétricas. Para além da componente técnica, o projeto contribuiu igualmente para o desenvolvimento de competências de planeamento, colaboração, controlo de versões e acompanhamento do trabalho em equipa, aspetos evidenciados ao longo da gestão dos sprints e da utilização de ferramentas como Bitbucket e Jira. Assim, considera-se que o trabalho realizado constituiu uma experiência relevante e enriquecedora, tanto do ponto de vista académico como do ponto de vista prático.

10 Bibliografia

- **Espressif Systems.** *ESP32 Series Datasheet*. Disponível em: <https://www.espressif.com/en/products/socs/esp32>. Consultado para compreender o funcionamento do microcontrolador ESP32 e as suas capacidades de comunicação.
- **Espressif Systems.** *ESP-IDF Programming Guide*. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. Utilizado para consulta de informações sobre programação e comunicação do ESP32.
- **Digilent.** *Basys 2 FPGA Board Reference Manual*. Disponível em: <https://digilent.com/reference/programmable-logic/basys-2/start>. Consultado para compreender a arquitetura e funcionamento da placa Basys2 utilizada no projeto.
- **Digilent.** *Basys2 Board Reference Manual (PDF)*. Disponível em: https://digilent.com/reference/_media/basys2:basys2_rm.pdf. Utilizado para obter informações técnicas sobre os pinos de entrada/saída e ligações da placa.
- **Arduino.** *Arduino Documentation*. Disponível em: <https://docs.arduino.cc/>. Utilizada para apoio no desenvolvimento de firmware e comunicação com microcontroladores.
- **Git.** *Git Documentation*. Disponível em: <https://git-scm.com/docs>. Consultado para compreender os procedimentos de controlo de versões utilizados no projeto.
- **Atlassian.** *Bitbucket Documentation*. Disponível em: <https://support.atlassian.com/bitbucket-cloud/>. Utilizado para compreender a gestão de repositórios e deployment do código do projeto.